A private trading community

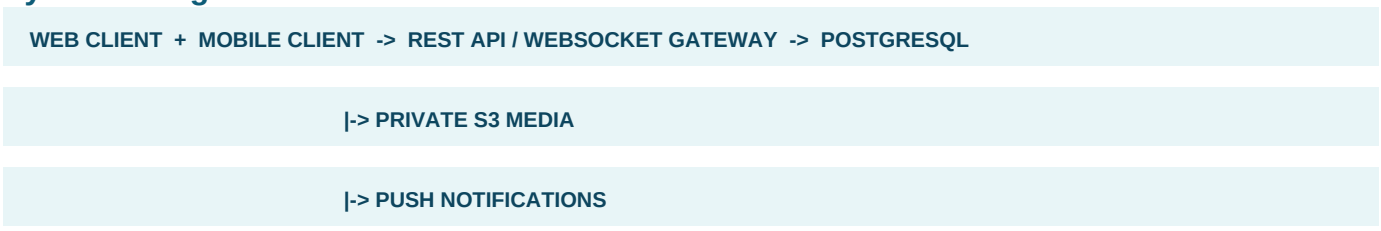
Architecture, delivery plan, and proposed database structure

Decision summary: extend the existing PostgreSQL and Express foundation into one member platform for web and mobile. Keep staff administration, signal publishing, member support, and private result review as separate permissioned workflows.

Product shape

Members follow only channels assigned to them, read trader posts, react, submit trade results privately, and start a one-to-one support conversation. Traders publish text, structured signals, and voice notes. Staff permissions are granted by a super admin and scoped to their responsibilities.

System at a glance



The API is the authority for identity, access checks, message writes, and signed media URLs. Web and mobile use the same endpoints and permission rules. PostgreSQL stores durable application data; S3 stores audio and other uploaded files.

Product decisions captured

- Signal edits show an Edited marker; preserve prior versions for staff audit.
- Results live in a separate staff-only review workflow, not a member-visible channel.
- Each support conversation has an assigned agent and a reassignment history.
- Plan both responsive web and mobile clients against a shared backend.

What exists today

This review is based on the project schema initializer and route code. It describes the schema the application creates and queries. The live database was not queried, so externally managed CRM tables and deployed-only migrations still need a read-only inventory before implementation.

Existing entities in the application schema

- portal_admins: staff credentials, admin ID, name, email, active state, and admin type.
- portal_admin_types, portal_admin_type_powers, portal_admin_powers: reusable admin profiles and powers.
- channels: channel metadata and creator fields; channel_admins maps staff to channels.
- channel_members: maps students to channels with uniqueness per channel and student.
- channel_messages: text/title, sender fields, edit timestamp, soft deletion, and media metadata.
- channel_message_reads and channel_message_admin_reads: read tracking for students and staff.
- superkandle_plans and superkandle_members: existing membership/payment-related feature.

The schema references students, courses, batches, and enrolments as existing CRM data. This project initializer does not create the students table; inspect the live schema and ownership of those tables before changing them.

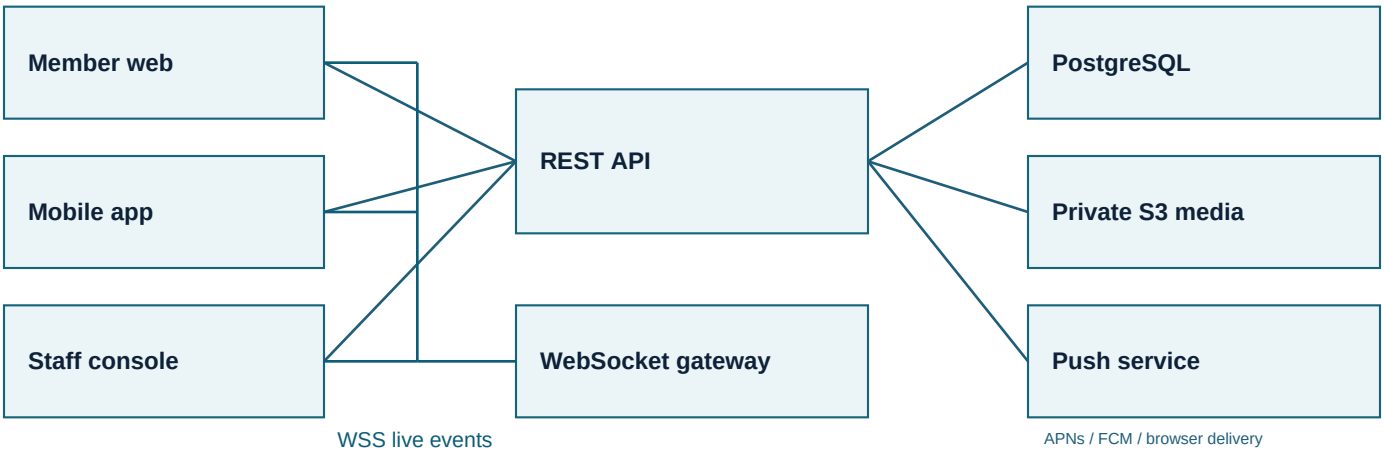
Useful behavior already present

- Express API, PostgreSQL connection pool, bearer-token authentication for staff, and a WebSocket channel layer.
- Channel access assignments, admin powers, channel message posting, read tracking, media upload, and message edits with edited_at.

Gaps for the requested product

- No app-side member login flow was confirmed in the inspected backend routes.
- No reaction, support conversation, support assignment, result submission, or message edit-history tables were found in the app schema initializer.
- Uploads currently use local disk and public /uploads serving; this needs private S3 objects and access-checked signed URLs.
- Super admin credentials are environment-based in the current login route; plan a managed super-admin account and stronger session controls before launch.

How the clients and services connect



Web and mobile share one authenticated API and data model. REST handles history, membership, message writes, support, and result review. WebSockets deliver authorized live channel and support events; the server rechecks authorization at subscription and delivery.

The API writes durable records to PostgreSQL and stores media in a private S3 bucket. A notification worker sends push notifications after committed events. Push payloads should contain only a safe preview; the client fetches protected content from the API after opening.

Each client uses both HTTPS and WSS. The WebSocket gateway is part of the backend trust boundary, not a direct client-to-database or client-to-S3 connection.

Services and trust boundaries

Application components

- Staff web console: manage users, roles, channel access, signals, support queue, result review, and audit events.
- Member web and mobile clients: channel feed, signal detail, reactions, result submission, support chat, and notifications.
- Backend API: authentication, authorization, REST reads/writes, validation, audit logging, media URL issuance, and notification events.
- WebSocket gateway: deliver authorized channel and support events; re-check access on connect, subscribe, and relevant access changes.
- PostgreSQL: accounts, permissions, messages, assignments, results, reactions, support history, and audit trail.
- Private S3 bucket: voice notes and other media. Store object key, content type, size, duration, and owner in PostgreSQL.
- Push notification service: notify on new signals, support replies, and assignment changes, respecting user preferences.

Authorization rules to enforce server-side

- Member reads require an active account and active channel membership; never trust a client-provided channel ID alone.
- Only assigned traders/staff with send_signals and channel scope can publish. Only the author or authorized supervisor can edit; edits create immutable history.
- Result submissions are visible only to the submitting member and authorized staff. Keep them out of member channel queries and broad WebSocket rooms.
- Support messages are visible to the conversation member and assigned agent; supervisors may receive explicit manage_support_all scope.
- S3 objects stay private. Issue short-lived signed URLs only after the same resource access check used for the message or result.
- When channel access is revoked, reject subsequent API reads, media URL requests, and WebSocket delivery immediately.

Operational baseline

Use database migrations rather than startup-time schema mutation for new production features; add backups, structured logs, rate limits, upload limits, retention rules, and monitoring before broad rollout.

Core tables to add or evolve

Retain the working channel and CRM entities where possible. Introduce explicit member identity and status only after confirming whether the CRM students table already provides those fields and authentication.

Accounts and access

- members (or CRM students): id, auth_subject/password_hash, status, created_at, last_seen_at. Do not duplicate student PII if CRM remains the source of truth.
- permission_catalog: key, description, scope_type. Seed powers such as create_channels, assign_channel_access, send_signals, manage_support, review_results, and view_channel_people.
- portal_admins and admin type tables: retain; add created_by_admin_id, updated_by_admin_id, and explicit account lifecycle as needed.
- channel_members: evolve with granted_by_admin_id, granted_at, revoked_at, and optional source/reason; unique active membership per member/channel.
- channel_admins: retain as channel scope assignment; store assignment actor and timestamps if audit detail is needed.

Channels and signals

- channels: retain; add channel_type (signals/general), visibility/status, and optional settings JSONB. Enforce staff-only result visibility in dedicated tables, not a flag alone.
- channel_messages: retain; add message_type (text/signal/voice), signal_data JSONB, updated_by_portal_admin_id, and version number if useful.
- channel_message_edits: id, message_id, editor_portal_admin_id, old_title, old_body, old_signal_data JSONB, edited_at. Insert old version and update message in one transaction.
- channel_message_media: id, message_id, object_key, media_type, mime_type, byte_size, duration_ms, original_name, created_at. Replace public local paths with private S3 keys.
- channel_message_reactions: id, message_id, member_id, reaction, created_at; unique (message_id, member_id, reaction).
- Retain channel_message_reads and channel_message_admin_reads for unread and seen indicators.

Support, results, and audit

Support conversations

- support_conversations: id, member_id, status (open/pending/resolved), assigned_agent_id, subject, created_at, updated_at, last_message_at, closed_at.
- support_messages: id, conversation_id, sender_kind, member_id or portal_admin_id, body, created_at, edited_at, deleted_at. Use a check constraint so exactly one sender identity matches sender_kind.
- support_assignments: id, conversation_id, agent_id, assigned_by_admin_id, assigned_at, unassigned_at, reason. This preserves the assignment history while the current agent remains on support_conversations.
- support_message_reads: message_id, viewer identity, seen_at, with uniqueness per message/viewer.

Private trade results

- trade_results: id, member_id, optional source_message_id, instrument, direction, entry_price, exit_price, outcome, pnl_amount, pnl_percent, notes, submitted_at, review_status, reviewed_by_admin_id, reviewed_at.
- trade_result_media: id, trade_result_id, object_key, mime_type, byte_size, created_at. Keep S3 private and issue signed URLs after staff/member ownership checks.

Use numeric columns with explicit precision for prices and P&L. Store currency and timezone assumptions if the business needs them. Validate all fields server-side; never infer a result outcome from a screenshot.

Audit and notification records

- admin_audit_log: id, actor_portal_admin_id, action, target_type, target_id, before_json, after_json, request_id, created_at. Record permission changes, channel access changes, edits, result reviews, and support reassignments.
- device_tokens: member_id, platform, token, last_seen_at, disabled_at; unique active token.
- notification_outbox: id, event_type, recipient_member_id, payload_json, created_at, processed_at, attempt_count. Use an outbox worker so database commits and notifications do not drift.

Important relationships and indexes

Every message belongs to one channel; every reaction belongs to one message and member; every support message belongs to one conversation; every result belongs to one member. Index channel_messages(channel_id, created_at DESC, id DESC), channel_members(member_id, channel_id), support_conversations(assigned_agent_id, status, last_message_at DESC), support_messages(conversation_id, created_at), and trade_results(submitted_at DESC, review_status).

Use foreign keys and unique constraints for ownership and deduplication. Prefer soft deletion or status transitions where audit/history matters. Apply migrations in small, reversible steps.

A safe path to the first release

Phase 0 - verify and baseline

- Inventory live PostgreSQL tables, columns, indexes, constraints, and row counts using a read-only account; identify CRM ownership of students and auth.
- Confirm the member authentication approach, account lifecycle, channel access source, result fields, retention policy, and notification needs.
- Write versioned migration and rollback scripts; capture a database backup before schema changes.

Phase 1 - member access and secure media

- Implement member login/session refresh and server-side channel authorization for REST and WebSockets.
- Build private S3 upload flow: API authorizes upload, returns short-lived upload URL, records object metadata, and issues download URLs after access checks.

Phase 2 - signal publishing

- Add structured signal template and free text/voice messages, scoped trader permissions, edit history, Edited marker, reactions, and live updates.

Phase 3 - assigned support

- Add member-to-support conversations, staff queue, claim/assign/reassign flow, message history, unread state, and supervisor access.

Phase 4 - private results

- Add member submission form, private attachments, staff-only queue, review status, and audit trail. Enforce privacy in every API and socket query.

Phase 5 - web/mobile launch readiness

- Complete responsive member web and mobile clients; push notifications; backups/restore drill; observability; abuse controls; accessibility; staged rollout.

Release acceptance checks

- A member cannot read or fetch media for an unassigned channel, another member result, or another member support chat.
- Permission changes and access revocation apply to API and WebSocket delivery without trusting stale client state.
- Signal edits show Edited and preserve the prior version; only permitted users can edit.
- A support thread has one current assigned agent and a queryable reassignment history.
- Result submissions remain staff-only unless an explicit future product decision changes that policy.